

Evaluación 2 — HTML + CSS + JS

- **Módulo 1 · Diplomado Fullstack V1 2026 · IPSS**
- **Profesor:** Gonzalo Fleming
- **Fecha:** Desde el 21-05-2026 hasta las 23:59 del 28-05-2026
- **Modalidad:** Grupal (3-4 estudiantes) · Trabajo con Git

Contexto general

Parámetro	Valor
Stack	HTML5, CSS3 + Bootstrap 5 (CDN), JS vanilla (sin build tools)
Base	Se construye sobre el sitio entregado en Evaluación 1

Esta evaluación no parte de cero. El objetivo es evolucionar el proyecto de la Evaluación 1 agregando funcionalidad JavaScript real. No se reescribe el sitio, se mejora.

Objetivos de aprendizaje evaluados

- **JavaScript:** manipulación del DOM, eventos, validación, fetch, asincronía básica, localStorage.
- **Refactor sobre código existente:** mejorar el sitio de Evaluación 1, no reescribirlo.
- **Git en escenario realista:** conflictos sobre código que ya existe, branches que dependen de otras.
- **Calidad:** accesibilidad, performance básica, código limpio.

Evolutivos requeridos — mínimo 4 de 6

El grupo debe implementar **al menos 4** de los siguientes evolutivos. Cada evolutivo corresponde a 1 PR (o 1 set de PRs relacionadas). Cada miembro debe ser dueño de al menos 1 evolutivo.

#	Evolutivo	Detalle técnico mínimo
E1	Validación de formulario en JS	Validar campos en submit, mostrar errores inline, prevenir envío si hay errores. Sin librerías.
E2	Consumo de API pública con fetch	Llamar a una API real, renderizar resultados en el DOM dinámicamente, manejar estado de carga y error.
E3	Filtro/búsqueda dinámica	Input que filtra una lista en tiempo real (DOM o sobre datos del fetch).
E4	Persistencia con localStorage	Favoritos, modo oscuro, carrito, o similar. Persiste al recargar la página.
E5	Componente interactivo	Modal, carrusel, acordeón, tabs, menú móvil con animación. Sin librerías.

#	Evolutivo	Detalle técnico mínimo
E6	Mejoras de calidad	Lighthouse ≥ 85 en Performance y Accessibility, navegación 100% por teclado, mejoras de SEO básico (meta tags, og tags).

Requisitos técnicos

Requisito	Detalle
JS sin frameworks	Vanilla JS. Nada de jQuery, React, Vue, etc.
Código modular	Separar JS en módulos por funcionalidad (<code>js/validacion.js</code> , <code>js/api.js</code> ...)
Sin errores en consola	Cargar el sitio no debe arrojar errores ni warnings críticos en Dev-Tools.
Async correcto	<code>fetch</code> con <code>async/await</code> o <code>.then()</code> , manejo de errores con <code>try/catch</code> o <code>.catch()</code> .
Mantener responsive	Todo lo que funcionaba en Eval 1 sigue funcionando.

Entregables

1. Repositorio actualizado con:

- Mínimo **4 PRs nuevas** (post Evaluación 1) mergeadas a main.
- Cada PR resuelve un evolutivo y tiene review aprobada.
- Tag **v1.0** apuntando al commit final de Evaluación 1.
- Tag **v2.0** apuntando al estado final de Evaluación 2.

```
# Cómo crear un tag
git tag v2.0
git push origin v2.0
```

2. README actualizado con:

- Sección “Evolutivos implementados” (descripción de cada uno).
- Sección “Decisiones técnicas” (qué API eligieron y por qué, cómo organizaron el JS, etc.).

3. CHANGELOG.md simple con los cambios entre v1.0 y v2.0.

Criterios de aprobación mínima

Si NO se cumple alguno de estos puntos, la evaluación se reprueba **sin entrar a rúbrica**:

- ☐ Mínimo 4 evolutivos implementados y funcionales.
- ☐ El sitio se carga sin errores en consola.
- ☐ Cada miembro hizo merge de al menos 1 PR en esta etapa.
- ☐ Lo entregado en Evaluación 1 sigue funcionando (no hay regresiones).

Rúbrica

Cada criterio se evalúa de **1 a 7** (escala chilena). Promedio ponderado.

Criterio	Peso	1-3 — Insuficiente	4-5 — Suficiente	6-7 — Sobresaliente
Cantidad y calidad de evolutivos	25%	Menos de 4 evolutivos o no funcionan	4 evolutivos funcionales básicos	5+ evolutivos con detalles cuidados
JS limpio y modular	20%	Todo en un archivo, código spaghetti, o interpola data externa sin sanitizar (XSS)	Separado en archivos por tema, sin vulnerabilidades XSS evidentes	Modular, funciones puras donde aplica, nombres claros, data externa siempre saneada (<code>textContent</code> o <code>escapeHTML</code>)
Manejo de async y errores	10%	No maneja errores, <code>race conditions</code>	Maneja errores básicos	Estados <code>loading</code> , <code>error</code> , <code>success</code> con UX cuidada
Git en escenario evolutivo	15%	Conflictos sin resolver, force push descontrolado	PRs OK pero sin tags ni changelog	Tags v1/v2, changelog, historial limpio
Code review entre pares	10%	“LGTM (+1)” en todo	Algunos comentarios sustantivos	Reviews que detectan bugs o sugieren mejoras
No hay regresiones de Eval 1	10%	Lo de antes está roto	Funciona pero algo se degradó	Todo de Eval 1 sigue intacto o mejoró
Documentación	5%	No se sabe qué hicieron	Lista los evolutivos	Explica decisiones técnicas y trade-offs

APIs públicas sugeridas para E2

API	URL	Útil para
DummyJSON	https://dummyjson.com/	Productos, usuarios, recetas
RestCountries	https://restcountries.com/	Datos de países
PokeAPI	https://pokeapi.co/	Catálogo de Pokémon
TheMealDB	https://www.themealdb.com/api.php	Recetas de comida
OpenLibrary	https://openlibrary.org/developers/api	Libros

Recursos

JavaScript

- JavaScript.info (en español): <https://es.javascript.info/>
- MDN — fetch: https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch
- MDN — localStorage: <https://developer.mozilla.org/es/docs/Web/API/Window/localStorage>

- MDN — async/await: <https://developer.mozilla.org/es/docs/Learn/JavaScript/Asynchronous/Pr omises>

Bootstrap 5

- Documentación oficial: <https://getbootstrap.com/docs/5.3/>
- Iconos: <https://icons.getbootstrap.com/>

Herramientas de calidad

- W3C Validator: <https://validator.w3.org/>
- Lighthouse (en Chrome DevTools — pestaña “Lighthouse”)
- WAVE (accesibilidad): <https://wave.webaim.org/>

Apéndice — CHANGELOG.md esqueleto

Crear este archivo en la raíz del repo:

```
# Changelog

## [v2.0] – 2026-XX-XX

### Agregado

- E1: Validación de formulario de contacto con errores inline
- E2: Consumo de API [nombre] con renderizado dinámico de resultados
- E4: Modo oscuro persistente con localStorage

### Mejorado

- Performance general (Lighthouse: Performance 88, Accessibility 92)
- Organización del JS en módulos separados

## [v1.0] – 2026-XX-XX

- Entrega inicial: sitio estático con Bootstrap 5 + CSS custom + eventos básicos
```

Apéndice — Estructura JS sugerida para Evaluación 2

```
js/
├─ main.js           ← punto de entrada, inicializa módulos
├─ validacion.js     ← lógica de validación de formularios (E1)
├─ api.js            ← fetch y manejo de datos externos (E2)
```

```
└─ filtro.js      ← búsqueda/filtro dinámico (E3)
└─ storage.js     ← localStorage: favoritos, modo oscuro, etc. (E4)
```

Ejemplo de `js/api.js` con manejo correcto de `async/errores`:

```
export async function fetchProductos(categoria) {
  const url = `https://dummyjson.com/products/category/${categoria}`

  try {
    const res = await fetch(url)
    if (!res.ok) throw new Error(`Error ${res.status}`)
    const data = await res.json()
    return data.products
  } catch (err) {
    console.error('Error al obtener productos:', err)
    return []
  }
}
```

Ejemplo de `js/main.js` renderizando con `createElement` (práctica segura):

```
import { fetchProductos } from './api.js'

function crearTarjeta(producto) {
  const card = document.createElement('div')
  card.className = 'card'

  const titulo = document.createElement('h3')
  titulo.textContent = producto.title // textContent, no innerHTML

  const precio = document.createElement('p')
  precio.textContent = `${producto.price}`

  card.appendChild(titulo)
  card.appendChild(precio)
  return card
}

async function renderProductos(categoria) {
  const contenedor = document.getElementById('productos')
  contenedor.textContent = 'Cargando...'

  const productos = await fetchProductos(categoria)

  contenedor.textContent = ''
}
```

```

if (productos.length === 0) {
  contenedor.textContent = 'No se encontraron resultados.'
  return
}

productos.forEach((p) => contenedor.appendChild(crearTarjeta(p)))
}

document.addEventListener('DOMContentLoaded', () => {
  renderProductos('smartphones')
})

```

Alternativa con innerHTML + escapeHTML (también válida):

Si prefieres usar template literals con innerHTML, debes envolver toda data externa en una función de sanitización. Nunca interpolas strings de APIs directamente.

```

function escapeHTML(str) {
  return String(str)
    .replaceAll('&', '&amp;')
    .replaceAll('<', '&lt;')
    .replaceAll('>', '&gt;')
    .replaceAll('"', '&quot;')
    .replaceAll("'", '&#39;')
}

function crearTarjetaHTML(producto) {
  return `
    <div class="card">
      <h3>${escapeHTML(producto.title)}</h3>
      <p>${escapeHTML(producto.price)}</p>
    </div>
  `
}

async function renderProductos(categoria) {
  const contenedor = document.getElementById('productos')
  contenedor.textContent = 'Cargando...'

  const productos = await fetchProductos(categoria)

  if (productos.length === 0) {
    contenedor.textContent = 'No se encontraron resultados.'
    return
  }
}

```

```
contenedor.innerHTML = productos.map(crearTarjetaHTML).join('')
}
```

Nota sobre XSS: al insertar datos de APIs externas, nunca interpolas directamente con `innerHTML`. Tienes dos caminos válidos en producción:

1. **textContent / createElement** — seguro por defecto, el navegador nunca interpreta el contenido como HTML.
2. **innerHTML con escapeHTML** — más expresivo con template literals, pero requiere disciplina: cada dato externo debe pasar por la función de sanitización.

El error a evitar es mezclar `innerHTML` con data sin escapar. Si en tu PR usas `innerHTML`, el code review debe verificar que toda interpolación esté saneada. En proyectos reales se suele usar una librería como `DOMPurify`; para esta evaluación basta con la función `escapeHTML` mostrada arriba.